

Key Takeaways

1 Transfer Learning Is the Default

In practice, you almost never train from scratch. Pre-trained CNNs provide generic features (edges, textures, shapes) that transfer to new tasks. Feature extraction (freeze all, train new head) works for small similar-domain datasets; fine-tuning (unfreeze some layers with small LR) works when you have more data or a different domain.

2 Object Detection = What + Where

Two main approaches: **Faster R-CNN** (two-stage: propose regions then classify — more accurate) and **YOLO** (single-shot: grid-based predictions in one pass — real-time capable). Both use pre-trained CNN backbones. YOLO dominates industry for speed; R-CNN family for high-stakes accuracy.

3 Segmentation = Pixel-Level Classification

Encoder-decoder architectures (FCN, U-Net) classify every pixel. The encoder downsamples to build semantic features; the decoder upsamples back to full resolution. Skip connections from encoder to decoder preserve spatial detail that downsampling destroys. Critical for medical imaging and autonomous driving.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** You have 300 labeled flower photos and want to classify 5 species. Should you use feature extraction or fine-tuning with a pre-trained ResNet? Why?
- Q2.** Why should you use a much smaller learning rate for pre-trained layers than for the new classification head during fine-tuning?
- Q3.** What problem does the Region Proposal Network (RPN) in Faster R-CNN solve compared to the sliding window approach?
- Q4.** In U-Net, what is the purpose of skip connections from encoder to decoder? What would happen without them?
- Q5.** A model trained on ImageNet photos performs poorly on X-ray images despite fine-tuning. What is this phenomenon called, and what's the simplest fix?

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Feature Extraction vs. From-Scratch Comparison

Load a pre-trained ResNet-18 and use it as a fixed feature extractor on CIFAR-10 (use only 1000 training images). Compare test accuracy after 10 epochs against a ResNet-18 trained from scratch on the same 1000 images. Plot both training curves on the same graph. How large is the gap? At what dataset size does training from scratch start to catch up?

Exercise 2: Layer-by-Layer Unfreezing Experiment

Using the same pre-trained ResNet-18 on CIFAR-10 (5000 images), compare three strategies: (a) freeze all except FC, (b) unfreeze layer4 + FC, (c) unfreeze layer3 + layer4 + FC. Use differential learning rates (10× smaller for pre-trained layers). Report test accuracy for each. Does unfreezing more layers always help?

Exercise 3: Object Detection on Custom Images

Load torchvision's pre-trained Faster R-CNN (ResNet-50 FPN backbone). Run inference on 5 images of your choice (download from the internet). For each image, visualize bounding boxes for detections with confidence > 0.5. Print the class names and confidence scores. Try one image with many objects and one with unusual/challenging content — when does the model fail?

Challenge Problem

Transfer Learning Decision Matrix — Empirical Validation

Build an experiment that validates the 2×2 decision framework (dataset size $\times$ domain similarity):

1. Use CIFAR-10 as “similar domain” (natural images) and a medical/satellite dataset (e.g., MedMNIST or EuroSAT) as “different domain”
2. For each domain, test with 500 images and 5000 images (small vs. large)
3. For each of the 4 cells, compare: feature extraction vs. fine-tune-last vs. fine-tune-all
4. Report a 4×3 results table (4 scenarios $\times$ 3 strategies)

Expected output: A table and a short paragraph explaining which strategy won in each cell and whether it matches the theoretical prediction.

Bonus: Add a “train from scratch” baseline to each cell. At what point does it become competitive?

- **Transfer Learning** — Yosinski et al., 2014, “How transferable are features in deep neural networks?” Definitive study on which layers transfer.
- **Faster R-CNN** — Ren et al., 2015, “Faster R-CNN: Towards Real-Time Object Detection with RPNs.” Read Sections 1–3.
- **YOLO** — Redmon et al., 2016, “You Only Look Once: Unified, Real-Time Object Detection.” Elegantly simple single-shot design.
- **U-Net** — Ronneberger et al., 2015, “U-Net: Convolutional Networks for Biomedical Image Segmentation.” Encoder-decoder + skip connections.

Online Resources & Tools

- **PyTorch Transfer Learning Tutorial** — Official guide with code for fine-tuning and feature extraction
- **Segment Anything (SAM)** — Meta’s universal segmentation model, great for experimentation
- **Ultralytics YOLOv8** — State-of-the-art YOLO implementation, easy to run on custom data
- **Hugging Face Model Hub** — Thousands of pre-trained vision models for various tasks
- **torchvision.models.detection** — Pre-trained Faster R-CNN, FCOS, RetinaNet ready to use
- **Roboflow** — Tools for labeling and training custom detection/segmentation models

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about “freezing layers”?** `param.requires_grad = False` means the layer still produces features in the forward pass, but its weights never update during backprop.
- **Fuzzy on learning rate schedules?** Revisit Session 4. Cosine decay starts high and smoothly decreases. For fine-tuning, use 10–100× smaller LR than training from scratch.
- **Not sure how upsampling works?** Transposed convolution or bilinear interpolation + regular conv. The reverse of pooling — expanding spatial dimensions back up.
- **Confused about IoU?** $\text{IoU} = \text{area of overlap} / \text{area of union}$ between predicted and ground-truth boxes. $\text{IoU} > 0.5$ is typically a “correct” detection.

Answers — Test Your Understanding

- A1.** Feature extraction. 300 images is very small + flowers are similar to ImageNet. Freeze ResNet, train only a new FC head. Fine-tuning risks overfitting with so little data.
- A2.** A large LR destroys pre-trained features in the first few updates (“catastrophic forgetting”). A small LR adapts features gently without erasing useful ImageNet knowledge.
- A3.** Sliding window tests millions of positions/scales, each needing a forward pass. RPN shares the backbone features and proposes only ~2000 regions — orders of magnitude faster.
- A4.** Skip connections pass high-resolution spatial details (edges, boundaries) directly to the decoder. Without them, the decoder only has the compressed bottleneck and produces blurry boundaries.
- A5.** Domain shift. Simplest fix: fine-tune on a small labeled set from the target domain (even 100–500 X-ray samples helps significantly).

Hands-on notebook: `transfer_learning_notebook.ipynb`

Instructor: Priyansh Saxena